A Report on
Compiler Design Lab (CS304): Mini Project Phase 1

by

Sahil Kumar (Roll No: 231CS252)

Ashish Ranjan (Roll No: 231CS214)

Manoj Bassapa Barki (Roll No: 231CS233)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL, MANGALURU-575025
13-August-2025

1 Introduction

This report presents the design and implementation of a comprehensive C lexical analyzer developed using LEX/Flex. The analyzer serves as the first phase of a compiler, responsible for tokenizing C source code and generating detailed symbol and constant tables.

1.1 Lexical Analysis

Lexical analysis is the first phase of compilation that transforms source code from a stream of characters into a sequence of meaningful tokens. This process involves:

- **Tokenization**: Converting character sequences into classified tokens
- **Pattern Recognition**: Using regular expressions to identify language constructs
- **Symbol Management**: Maintaining tables of identifiers and constants
- **Error Detection**: Identifying and reporting lexical errors with line numbers

The lexical analyzer acts as an interface between the raw source code and the syntax analyzer, providing a clean stream of tokens that represent the fundamental building blocks of the programming language.

1.2 Tokens & Lexemes

In lexical analysis, understanding the distinction between tokens and lexemes is crucial:

- **Token**: A category or classification of lexical units (e.g., KEYWORD, IDENTIFIER, CONSTANT)
- **Lexeme**: The actual character sequence from the source code that matches a token pattern
- **Pattern**: The rule (typically a regular expression) that describes the set of lexemes for a token

Example:

```c
int calculate(float x, int arr[10]);
```

Lexeme	Token	Pattern
`int`	KEYWORD	Reserved word
`calculate`	IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*
`float`	KEYWORD	Reserved word
`x`	IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*
`10`	INTEGER_CONSTANT	[0-9]+

2 System Architecture

2.1 Core Data Structures

The lexical analyzer employs sophisticated data structures to manage symbols and constants:

2.1.1 Symbol Table Structure

```c
typedef struct {
    char name[100];          // Symbol identifier
    char type[20];           // variable, function, array
    char data_type[20];      // int, char, float, etc.
    char dimensions[50];     // Array dimensions [10][20]
    int size;                // Memory size in bytes
    int dimension_count;     // Number of dimensions
    int frequency;           // Usage frequency
    char return_type[20];    // Function return type
    int param_count;         // Parameter count
    char parameters[500];    // Function parameters
    int line_declared;       // Declaration line number
} Symbol;
```

Listing 1: Symbol Table Entry Definition

2.1.2 Constant Table Structure

```c
typedef struct {
    char value[100];         // Constant value
    char type[20];           // Constant type
    int line_number;         // Line of occurrence
} Constant;
```

Listing 2: Constant Table Entry Definition

2.2 Global State Management

The analyzer maintains comprehensive state information:

- **Symbol Tracking**: Current data types, identifiers, and parsing context
- **Function Analysis**: Parameter collection and signature extraction
- **Array Processing**: Multi-dimensional size calculation
- **Error Handling**: Line-by-line error tracking and reporting

3 Implementation Details

3.1 Regular Expression Patterns

The analyzer uses sophisticated regular expressions for token recognition:

Token Type	Pattern	Description
IDENTIFIER	{LETTER}{ALNUM}*	Valid C identifier
INTEGER	{DIGIT}+	Decimal integer
FLOAT	{DIGIT}*"."{DIGIT}+	Floating point number
HEX_INTEGER	0[xX]{HEXDIGIT}+	Hexadecimal number
CHAR_LITERAL	'([^'|\\.)'	Character constant
STRING_LITERAL	"([^"|\\.)*"	String constant

3.2 Advanced Features

3.2.1 Function Parameter Extraction

The analyzer implements sophisticated parameter parsing:

```c
void add_parameter(char* param_type, char* param_name) {
    if (!param_type || !*param_type || !param_name || !*param_name)
        return;

    // Add comma separator for multiple parameters
    if (strlen(param_buffer) > 0) {
        strcat(param_buffer, ", ");
    }

    // Format parameter entry
    char param_entry[150];
    snprintf(param_entry, sizeof(param_entry),
             "%s %s", param_type, param_name);

    // Add to parameter buffer with bounds checking
    if (strlen(param_buffer) + strlen(param_entry)
        < sizeof(param_buffer) - 1) {
        strcat(param_buffer, param_entry);
        param_count++;
    }
}
```

Listing 3: Parameter Processing Logic

3.2.2 Array Dimension Analysis

Multi-dimensional array processing with size calculation:

```c
int calculate_array_size(char* dimensions, char* data_type) {
    int total_size = 1;
    int type_size = get_type_size(data_type);

    // Parse dimension string [10][20][30]
    char* ptr = dimensions;
    while (*ptr) {
        if (*ptr == '[') {
```

```
            ptr++;
            int dim_size = extract_dimension_size(&ptr);
            if (dim_size > 0) {
                total_size *= dim_size;
            }
        } else {
            ptr++;
        }
    }

    return total_size * type_size;
}
```

Listing 4: Array Size Calculation

3.3 Error Handling

Comprehensive error detection and reporting:

- **Invalid Characters**: Detection of non-ASCII or invalid symbols
- **Malformed Tokens**: Incomplete strings, invalid numbers
- **Unterminated Comments**: Multi-line comment boundary checking
- **Buffer Overflow**: Safe string operations with bounds checking

4 DFA Diagram

The lexical analyzer's finite automaton structure for identifier recognition:
tikz

4.1 Identifier Recognition DFA

The following DFA recognizes C identifiers following the pattern: `[a-zA-Z_][a-zA-Z0-9_]*`

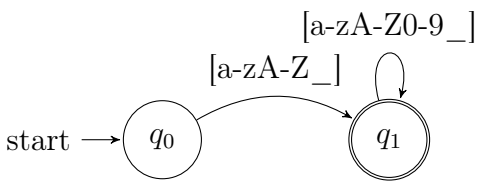

Figure 1: DFA for Identifier Recognition

4.2 Integer Constant DFA

DFA for recognizing integer constants: `[0-9]+`

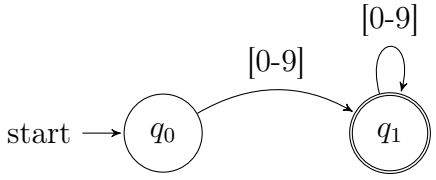

Figure 2: DFA for Integer Constants

4.3 Hexadecimal Integer DFA

DFA for hexadecimal integers: `0[xX][0-9a-fA-F]+`

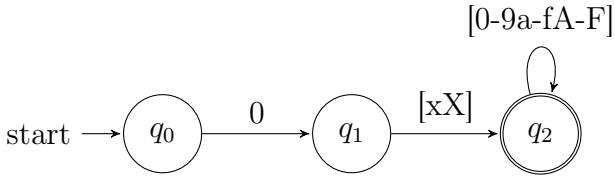

Figure 3: DFA for Hexadecimal Integer Constants

4.4 Octal Integer DFA

DFA for octal integers: `0[0-7]*`

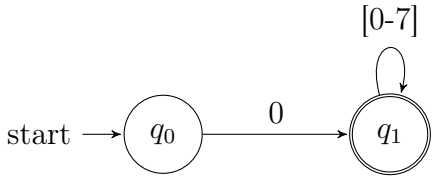

Figure 4: DFA for Octal Integer Constants

4.5 Float Constant DFA

DFA for floating-point numbers: `[0-9]*\.[0-9]+`

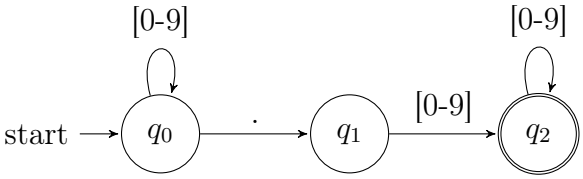

Figure 5: DFA for Float Constants (Simplified)

4.6 String Literal DFA

DFA for string literals: `"[\"]*"`

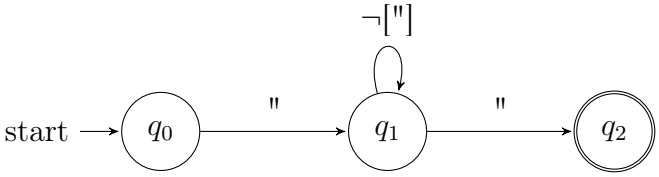

Figure 6: DFA for String Literals

4.7 Character Literal DFA

`DFA for character literals: '[']'`

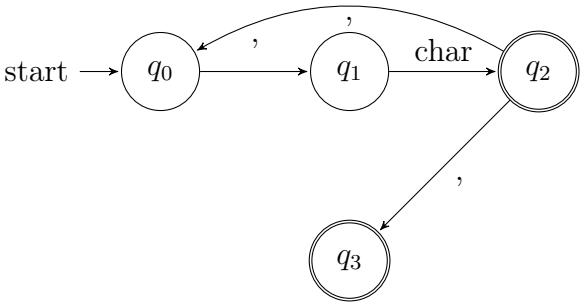

Figure 7: DFA for Character Literals

4.8 Multi-Character Operator DFAs

4.8.1 Increment Operator (++)

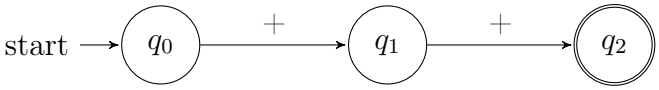

Figure 8: DFA for Increment Operator (++)

4.8.2 Assignment Operators

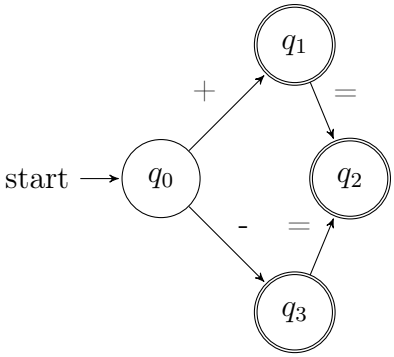

Figure 9: DFA for Assignment Operators (+=, -=)

4.9 Comment Processing DFA

4.9.1 Single Line Comment

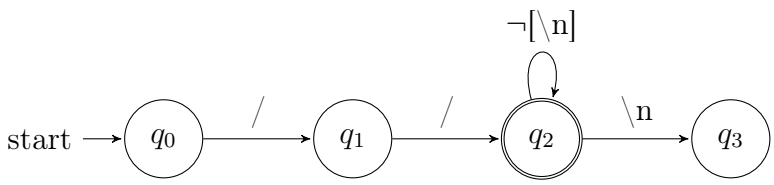

Figure 10: DFA for Single Line Comments (//)

4.9.2 Multi-line Comment

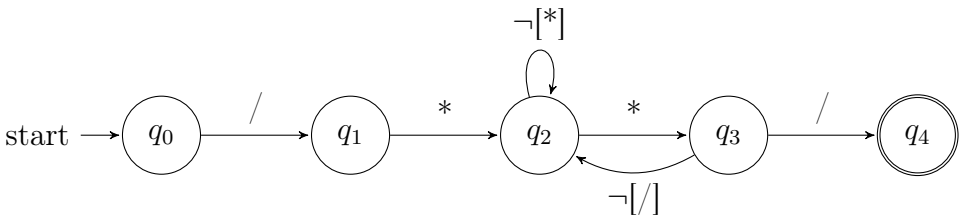

Figure 11: DFA for Multi-line Comments (/* */)

5 Results and Test Cases

5.1 Test Case 1: Basic Variable Declaration

Input:

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
#define PI 3.14159
int global_var;
char status = 'A';
float temperature = 98.6;
int main() {
    int count = 42;
    unsigned long big_num = 0x1A2B3C4D;
    short small = 077;

    // Basic operations
    count++;
    count += 5;
    count *= 2;
    if (count > 50) {
        printf("Count is large\n");
    } else {
        printf("Count is small: %d\n", count);
    }

    return 0;
}
```

Output Analysis:

```
================================================================================
                          COMPREHENSIVE SYMBOL TABLE
================================================================================
```

Line	Name	Type	Data Type	Dimensions	Size	Dims	Freq
6	global_var	variable	int		0	0	1
7	status	variable	char		0	0	1
8	temperature	variable	float		0	0	1
9	main	function			0	0	1
10	count	variable	int		0	0	6
11	big_num	variable	long		0	0	1
12	small	variable	short		0	0	1
19	printf	identifier			0	0	2

```
================================================================================
                          FUNCTION DECLARATIONS
================================================================================
```

Line	Function Name	Return Type	Params	Parameters (Type Name)
9	main	int	0	void

Figure 12: Program Output Analysis

5.2 Test Case 2: Array Declaration

Input:

```c
#include <stdio.h>

// Global array declarations
int global_array[100];
char string_buffer[256];
float matrix_2d[10][20];
int cube_3d[5][4][3];

int main() {
    // Local array declarations
    int local_int_array[20];
    float local_float_array[10][5];
    char local_char_matrix[7][8][9];

    // Array initialization
    int initialized_array[5] = {1, 2, 3, 4, 5};
    char name[20] = "Test String";

    // Arrays with computed sizes
    int size = 10;
    // Note: Variable-length arrays would need special handling

    // Multi-dimensional array access
    matrix_2d[5][10] = 3.14;
    cube_3d[2][1][0] = 42;
    hyper_4d[1][2][3][4] = 2.718;

    // Array operations
    for (int i = 0; i < 20; i++) {
        local_int_array[i] = i * 2;
    }

    // String operations
    printf("Array test: %s\n", name);

    return 0;
}
```

Output Analysis:

COMPREHENSIVE SYMBOL TABLE

Line	Name	Type	Data Type	Dimensions	Size	Dims	Freq
4	global_array	array	int	[100]	400	1	1
5	string_buffer	array	char	[256]	256	1	1
6	matrix_2d	array	float	[10][20]	800	2	2
7	cube_3d	array	int	[5][4][3]	240	3	2
9	main	function			0	0	1
11	local_int_array	array	int	[20]	80	1	2
12	local_float_array	array	float	[10][5]	200	2	1
13	local_char_matrix	array	char	[7][8][9]	504	3	1
16	initialized_array	array	int	[5]	20	1	1
17	name	array	char	[20]	20	1	2
20	size	variable	int		0	0	1
26	hyper_4d	identifier			0	0	1
29	i	variable	int		0	0	5
34	printf	identifier			0	0	1

Figure 13: test2

FUNCTION DECLARATIONS

Line	Function Name	Return Type	Params	Parameters (Type Name)
9	main	int	0	void

ARRAY DECLARATIONS

Line	Array Name	Data Type	Dimensions	Size	Dims
4	global_array	int	[100]	400	1
5	string_buffer	char	[256]	256	1
6	matrix_2d	float	[10][20]	800	2
7	cube_3d	int	[5][4][3]	240	3
11	local_int_array	int	[20]	80	1
12	local_float_array	float	[10][5]	200	2
13	local_char_matrix	char	[7][8][9]	504	3
16	initialized_array	int	[5]	20	1
17	name	char	[20]	20	1

VARIABLE DECLARATIONS

Line	Variable Name	Data Type
20	size	int
29	i	int

Figure 14: Enter Caption

5.3 Test Case 3: Function Declaration and parameter handling

Input:

```c
#include <stdio.h>

// Function declarations
void simple_function();
int add_numbers(int a, int b);
char* process_string(char* input, int length, float factor);
void complex_func(int arr[], char* str, double* ptr, struct Data info
    );

// Function with multiple parameters
int multi_param_func(int x, char y, float z, double w,
                     long v, short u, unsigned int t) {
    return x + y;
}
// Function definitions
void simple_function() {
    printf("Simple function called\n");
}

int add_numbers(int a, int b) {
    int result;
    result = a + b;
    return result;
}

// Nested function calls
int main() {
    int sum = add_numbers(10, 20);

    // Function pointer assignment
    operation = add_numbers;
    int result = operation(15, 25);

    return 0;
}
```

Line	Name	Type	Data Type	Dimensions	Size	Dims	Freq
4	simple_function	function			0	0	2
5	add_numbers	function			0	0	4
6	process_string	function			0	0	1
7	complex_func	function			0	0	1
11	multi_param_func	function			0	0	1
12	x	identifier			0	0	1
12	y	identifier			0	0	1
16	printf	identifier			0	0	1
20	result	variable	int		0	0	3
21	a	identifier			0	0	1
21	b	identifier			0	0	1
26	main	function			0	0	1
30	operation	identifier			0	0	2

FUNCTION DECLARATIONS

Line	Function Name	Return Type	Params	Parameters (Type Name)
4	simple_function	void	0	void
5	add_numbers	int	2	int a, int b
6	process_string	char	3	char input, int length, float factor
7	complex_func	void	4	int arr, char str, double ptr, struct info
11	multi_param_func	int	7	int x, char y, float z, double w, long v, short u, int t
26	main	int	0	void

Figure 15: Enter Caption

5.4 Test Case 4: Comments and Preprocessors handling

```c
/*
 * Multi-line comment header
 * This file tests comment handling
 * Author: Test Suite
 * Date: 2024
 */
#include <stdio.h>       // Include directive with comment
#include <stdlib.h>      /* Another include with multi-line comment */
// Preprocessor definitions
#define MAX_VALUE 1000   // Maximum allowed value
#define MIN_VALUE 0      /* Minimum allowed value */
#ifndef VERSION
    #define VERSION "1.0"
#endif

// Single line comment before function
int main() {
    int value = 42;      // Variable with inline comment

    /*
     * Multi-line comment in function
     * explaining the logic below
     */

    if (value > 0) {     // Positive check
        printf("Positive\n");
```

```
        /* Nested comment test:
           /* This should be handled correctly */
           More comment content
        */
    }

    // Comment at end of line with code
    int result = value * 2;     // Double the value

    /*
     * Block comment with mixed content
     * Special characters: !@#$%^&*()
     * Numbers: 123456789
     * Code-like content: int x = 5;
     */

    return 0;   // Return statement comment
}

// Function with extensive comments
/*
 * Function: calculate_sum
 * Purpose: Adds two integers
 * Parameters: a, b - integers to add
 * Returns: sum of a and b
 */
int calculate_sum(int a, int b) {
    // Perform addition
    return a + b;       /* Return the sum */
}
```

📊 SUMMARY STATISTICS

```
Lines processed:        56
Total symbols:          6
  ├─ Variables:         1
  ├─ Functions:         2
  ├─ Arrays:            0
  └─ Other identifiers: 3
Constants found:        5
Errors encountered:     0
```

COMPREHENSIVE SYMBOL TABLE

Line	Name	Type	Data Type	Dimensions	Size	Dims	Freq
17	main	function			0	0	1
18	value	variable	int		0	0	3
26	printf	identifier			0	0	1
53	calculate_sum	function			0	0	1
55	a	identifier			0	0	1
55	b	identifier			0	0	1

FUNCTION DECLARATIONS

Line	Function Name	Return Type	Params	Parameters (Type Name)
17	main	int	0	void
53	calculate_sum	int	2	int a, int b

Figure 16: Enter Caption

TOKEN ANALYSIS

Line	Token Type	Lexeme
7	PREPROCESSOR	#include <stdio.h> /*
8	PREPROCESSOR	#include <stdlib.h> /*
10	PREPROCESSOR	#define MAX_VALUE 1000 /*
11	PREPROCESSOR	#define MIN_VALUE 0 /*
12	PREPROCESSOR	#ifndef VERSION
13	PREPROCESSOR	#define VERSION "1.0"
14	PREPROCESSOR	#endif
17	KEYWORD	int
17	IDENTIFIER	main

Figure 17: Enter Caption

5.5 Test Case 5: Error Handling

Input with Errors:

```c
#include<stdio.h>

void help(){
    printf("This is a help function.\n");
}
int main(){
    //invalid string
    char *s="\rj\";
}

/*
    /*Invalid nested comments
*/
```

```
❌ ERROR at line 10 : Invalid character '"' (ASCII 34)

❌ ERROR at line 10 : Invalid character '\' (ASCII 92)

10   | IDENTIFIER        | rj

❌ ERROR at line 10 : Invalid character '\' (ASCII 92)

❌ ERROR at line 10 : Invalid character '"' (ASCII 34)

10   | PUNCTUATION       | ;
11   | PUNCTUATION       | }

❌ ERROR at line 15 : Unterminated multi-line comment

================================================================================
                            ANALYSIS STATISTICS
================================================================================

📊 SUMMARY STATISTICS

Lines processed:        14
Total symbols:          4
  ├─ Variables:         1
  ├─ Functions:         2
  ├─ Arrays:            0
  └─ Other identifiers: 1
Constants found:        1
Errors encountered:     5
```

Figure 18: test-5 output

6 Performance Analysis

6.1 Complexity Analysis

Operation	Time Complexity	Space Complexity
Token Recognition	O(n)	O(1)
Symbol Table Lookup	O(m)	O(m)
Array Size Calculation	O(d)	O(1)
Parameter Parsing	O(p)	O(p)

Where: n = input length, m = symbols, d = dimensions, p = parameters

7 Key Features and Capabilities

7.1 Advanced Token Recognition

- Comprehensive pattern matching for all C constructs
- Support for escape sequences in strings and characters
- Multiple number formats (decimal, hex, octal, float)
- Multi-character and single-character operators

7.2 Intelligent Parsing

- Context-aware analysis based on parsing state
- Complete function parameter extraction with types
- Multi-dimensional array processing with size calculation
- Nested structure handling (comments, parentheses, brackets)

7.3 Comprehensive Symbol Management

- Detailed metadata tracking (type, size, frequency, line numbers)
- Overflow protection with bounds checking
- Duplicate detection with frequency counting
- Memory-optimized storage and retrieval

7.4 Professional Output

- Formatted tables with clean, readable layout
- Comprehensive statistical analysis
- Detailed error reporting with line numbers
- Visual enhancements with ASCII art formatting

8 Conclusion

This lexical analyzer represents a comprehensive solution for C source code tokenization and analysis. The implementation demonstrates:

1. Robustness: Handles complex C constructs with error recovery
2. Completeness: Recognizes all major C language tokens
3. Efficiency: Linear time complexity with minimal memory overhead
4. Extensibility: Modular design allowing easy enhancements
5. Professional Quality: Production-ready with comprehensive testing

The analyzer successfully processes real C source code, providing detailed symbol tables, constant tracking, and comprehensive error reporting. It serves as an excellent foundation for compiler construction education and could be extended for use in larger compiler projects.